
Feature Implementation Guide

YvY Capital / Inteli

Technical documentation

Generated from the supplied Markdown source
12 August 2026

Contents

Feature 1 — Daily NAV and Portfolio Dashboard	3
Files you need	3
Recipe: fund NAV over time	3
Recipe: portfolio composition dashboard	3
Data quality notes for this feature	4
Feature 2 — Performance View: Returns and Risk-Return Indicators	4
Returns (fetched — returns_navps.csv)	4
Risk data that's fetched (real, ready to use)	4
Risk-return indicators you must calculate yourselves	5
Worked example (pseudocode, language-agnostic)	5
Feature 3 — Comparisons Across Yvy's Own Funds	6
Feature 4 — Benchmark Comparison Against Peers (not in this delivery)	6
Appendix A — Join keys at a glance	6
Appendix B — Parsing the JSON columns (stress_risk.csv, dv01.csv)	7

This document exists for one purpose: to save you time turning the CSVs in output/ into the DataApp features Yvy's TAPI asks for. DATA_DICTIONARY.md tells you what each column means; this document tells you **which columns to combine, how, and with what formula**, for each expected feature. Read DATA_DICTIONARY.md first if a column name here doesn't ring a bell.

Everything here is scoped to **this partial delivery** — INOA data for Yvy's own 14 funds only. Peer/benchmark data from ANBIMA is a separate, later delivery; Feature 4 below explains what that will add once it arrives.

Join key you'll use constantly: `fund_id`. It's the same anonymized fund (Fund A, Fund B, ...) across every single CSV in this delivery — every recipe below starts by filtering or grouping on it.

Feature 1 — Daily NAV and Portfolio Dashboard

What the TAPI asks for: a dashboard replacing the daily email — how big is each fund (NAV), and what does it currently hold.

Files you need

File	What it gives you
<code>funds.csv</code>	The fund picker/list (code, active status).
<code>fund_nav_snapshot.csv</code>	NAV in BRL at each real snapshot date.
<code>portfolio_holdings.csv</code>	Full holdings breakdown at each of those same snapshot dates.
<code>returns_navps.csv</code>	A daily index series — the key to filling the gaps between snapshots.

Recipe: fund NAV over time

`fund_nav_snapshot.csv` already gives you (`fund_id`, `snapshot_date`, `nav`) — plot it directly for a "NAV history" chart. But its dates are ~monthly checkpoints, not literally daily (see DATA_DICTIONARY.md's "Why some CSVs look small"). If your dashboard needs a NAV value for any date, not just the sampled ones, reconstruct it from `returns_navps.csv`'s daily index (`series_type = "fund"`, `accrued_return_pct`, confirmed rebased to 100 on the fund's own start date):

$$\text{nav}(\text{date}) = \text{nav}(\text{anchor_date}) \times \text{accrued_return_pct}(\text{date}) / \text{accrued_return_pct}(\text{anchor_date})$$

Pick any `anchor_date` that exists in both `fund_nav_snapshot.csv` and `returns_navps.csv` for that fund (there will be several, since both are pulled over the same real history) — the ratio cancels out the rebase-to-100 constant, so the result is a true BRL NAV for any date the index has a point for.

Recipe: portfolio composition dashboard

- 1 Pick the fund and a `snapshot_date` (default to the latest one present for that fund in `portfolio_holdings.csv`).
- 2 Filter `portfolio_holdings.csv` to (`fund_id`, `snapshot_date`) — every row left is one holding.

- 3 Group by `group_identifier` (stocks/bonds/cash/funds/provisions) and sum `nav_value` for an allocation pie chart; sum `nav_percentage` per group as a sanity check (should sum close to 1.0 across all groups for that snapshot, modulo rounding).
- 4 For a fixed-income drill-down, join on `instrument_id` against `bond_instruments.csv` to show ISIN/CETIP/maturity alongside each bond holding.

Data quality notes for this feature

- 1 `portfolio_holdings.csv` has several snapshot dates per fund, not one — always filter/group by `snapshot_date`, never assume one row per fund.
- 1 `quantity/instrument_id` are blank for cash and provisions rows — that's expected, not missing data (see `DATA_DICTIONARY.md`).
- 1 `bond_modified_duration.csv` and `external_debenture_data_raw.csv` are empty in this delivery (INOA account permissions) — a "duration" or "secondary market price" column on a bond drill-down isn't buildable from this delivery yet.

Feature 2 — Performance View: Returns and Risk-Return Indicators

What the TAPI asks for: per-fund performance, plus "risk-return indicators." Split below into what's fetched (real INOA data) vs. what you must calculate yourselves — INOA doesn't expose volatility or Sharpe/Sortino as raw numbers anywhere (confirmed by exploring every risk-related endpoint; see `contexto/data-catalog.md` §4 in the initiative's docs).

Returns (fetched — `returns_navps.csv`)

Same rebased-to-100 index as Feature 1. Two things you'll build from it:

- 1 **Cumulative return chart:** plot `accrued_return_pct` directly per (`fund_id`, `series_type`, `date`) — fund and benchmark (Yvy's own configured CDI-linked benchmark, e.g. "100% CDI + 0% a.a." — the real label, see `series_name`) on the same chart is exactly the "fund vs. benchmark" view the TAPI's own MVP language uses.
- 1 **Return between any two dates** (e.g. "return this month"):

$$\text{period_return_pct} = (\text{accrued_return_pct}(\text{date2}) / \text{accrued_return_pct}(\text{date1}) - 1) \times 100$$

- 1 **"% of CDI"** (the standard Brazilian way to describe a fund's performance) — compute each side's `period_return_pct` over the same window, then divide:

$$\text{pct_of_cdi} = \text{fund_period_return_pct} / \text{benchmark_period_return_pct} \times 100$$

Risk data that's fetched (real, ready to use)

Indicator	File	Notes
Drawdown	drawdown.csv	Already a daily fraction below peak — plot directly, no calculation needed.
Liquidity profile	liquidity_by_horizon.csv	% of NAV liquidatable per horizon (1/5/21/63/126/252 business days) — a bar chart per fund, "as of today" only (INOA doesn't expose liquidity history).
VaR/stress (computed)	stress_risk.csv	Parse risk_list_json — it's not flat columns. See "Parsing the JSON columns" below. Ignore var_mask_configs.csv for an actual number; that file is only the VaR configuration, not a result.
DV01 (rate sensitivity)	dv01.csv	Same deal — parse items_json.

Risk-return indicators you must calculate yourselves

Both use returns_navps.csv's daily index. Convert the index to a **daily return series** first:

```
daily_return_t = value(t) / value(t-1) - 1      # for consecutive business-day
points
```

1 Volatility (annualized standard deviation of daily returns):

```
volatility_annualized = stdev(daily_return) * sqrt(252)
```

(252 ≈ Brazilian business days/year — the standard convention for BRL fixed-income/multi-market funds; document whichever day-count you pick, since INOA doesn't hand you one.)

1 Sharpe ratio (using the CDI benchmark series as the risk-free proxy — a defensible choice since that's literally what CDI is used for in this market):

```
excess_return_t = fund_daily_return_t - cdi_daily_return_t
sharpe = mean(excess_return) / stdev(fund_daily_return) * sqrt(252)
```

There are variants of this formula (some divide by stdev(excess_return) instead of stdev(fund_daily_return)) — pick one, and say which in your own docs; INOA has no opinion here,

this is a modeling choice your team makes, not something to look up.

1 Sortino ratio (Sharpe's cousin, penalizing only downside volatility) — same excess_return_t series, but the denominator only uses days where excess_return_t < 0:

```
downside_deviation = stdev(excess_return_t where excess_return_t < 0)
sortino = mean(excess_return) / downside_deviation * sqrt(252)
```

Worked example (pseudocode, language-agnostic)

```
rows = returns_navps.filter(fund_id == X, series_type in ["fund", "benchmark"])
fund_series = rows.filter(series_type == "fund").sort_by(date)
cdi_series = rows.filter(series_type == "benchmark").sort_by(date)

fund_daily_returns = [fund_series[i].value / fund_series[i-1].value - 1 for i in 1..n]
cdi_daily_returns = [cdi_series[i].value / cdi_series[i-1].value - 1 for i in 1..n]
excess = fund_daily_returns - cdi_daily_returns # elementwise

volatility = stdev(fund_daily_returns) * sqrt(252)
sharpe = mean(excess) / stdev(fund_daily_returns) * sqrt(252)
```

Feature 3 — Comparisons Across Yvy's Own Funds

What the TAPI asks for: comparing Yvy funds against each other. **This needs no new data** — every file above already covers all 14 funds with the same `fund_id`/anonymized code, so this is purely a UI/aggregation feature on top of Features 1-2, not a new data source.

Two shapes worth building:

- 1 **Ranking table:** one row per fund, columns = whatever period return / volatility / Sharpe / current NAV you computed in Feature 2, sorted by whichever column the user picks. This is exactly a `GROUP BY fund_id` over the per-date series once you've computed the derived metrics.
- 1 **Small-multiples chart:** the same cumulative-return line chart from Feature 2, one line per fund, sharing an x-axis — lets a manager eyeball which fund is outperforming without a table.

Nothing here is blocked by anything in this delivery.

Feature 4 — Benchmark Comparison Against Peers (not in this delivery)

What the TAPI asks for: comparing each Yvy fund against comparable external market funds.

Out of scope for this delivery. INOA only has data for Yvy's own funds — it has no visibility into any other asset manager's fund. This feature needs a second, ANBIMA-sourced extract (a later phase of this same handoff — see `docs/initiatives/inteli-data-handoff/PLAN.md`, decisions 2-3), which will bring:

- 1 A real classification taxonomy (fund type + 3 category levels) to define what counts as a "peer" for any given Yvy fund.
- 1 A quota/return time series for peer funds, structurally similar to `returns_navps.csv` here, so the same period-return/volatility recipes above will carry over once that data lands.

If you want to start scaffolding the UI for this feature now, build it against a placeholder/mock peer dataset shaped like `returns_navps.csv` — the real ANBIMA extract, once delivered, is expected to slot into the same shape.

Appendix A — Join keys at a glance

```

funds.csv (fund_id) —┬─ fund_nav_snapshot.csv (fund_id, snapshot_date)
                    │┬─ portfolio_holdings.csv (fund_id, snapshot_date)
                    │ │┬─ instrument_id — bond_instruments.csv
                    │ │└─ (blocked)
                    │└─ bond_modified_duration.csv (instrument_id)
                    └─┬─ returns_navps.csv (fund_id, date)
                      │┬─ transactions_summary.csv (fund_id)
                      │┬─ cash_flow_daily.csv (fund_id, date)
                      │┬─ corporate_payments.csv (fund_id, ex_date)
                      │┬─ drawdown.csv (fund_id, date)
                      │┬─ liquidity_by_horizon.csv (fund_id, projection_days)
                      │┬─ stress_risk.csv (fund_id, date)
                      └─ dv01.csv (fund_id, date)

var_mask_configs.csv — tenant-wide, no fund_id; context only (see Feature 2).

```

Appendix B — Parsing the JSON columns (stress_risk.csv, dv01.csv)

risk_list_json and items_json are strings containing JSON, not flat columns — INOA's own docs never specify their structure precisely enough to safely split into invented columns, so they're passed through as-is (see DATA_DICTIONARY.md). In practice:

- 1 Read the CSV normally (both columns are string type — most CSV libraries won't auto-parse them).
- 2 JSON.parse() (or your language's equivalent) on that one cell.
- 3 For stress_risk.csv's risk_list_json: a single-element array (one result, for this row's own fund_id), containing instruments_risk[] (one entry per asset group, e.g. "Compromissadas"/"Caixa" — see the real shape documented in DATA_DICTIONARY.md), each with vals[].stress[] keyed by mask_id matching stress_mask_ids in the same row. sm_names inside gives you the scenario's display name if you want to label a chart axis with something more human than "mask 3".
- 4 For dv01.csv's items_json: an array of exposure units, each with riskFactorSymbol, duration, dv01Impact, etc. — several of these came back null for cash/repo positions in the funds checked (expected, not a bug — those instrument types have no meaningful duration).

If a row's success is false or risk_list_json/items_json is an empty string, there's no result for that fund/date — that's stress_risk.csv's honest way of saying "no stress mask ids were discoverable on this tenant" (see the file's own notes if that ever happens again).